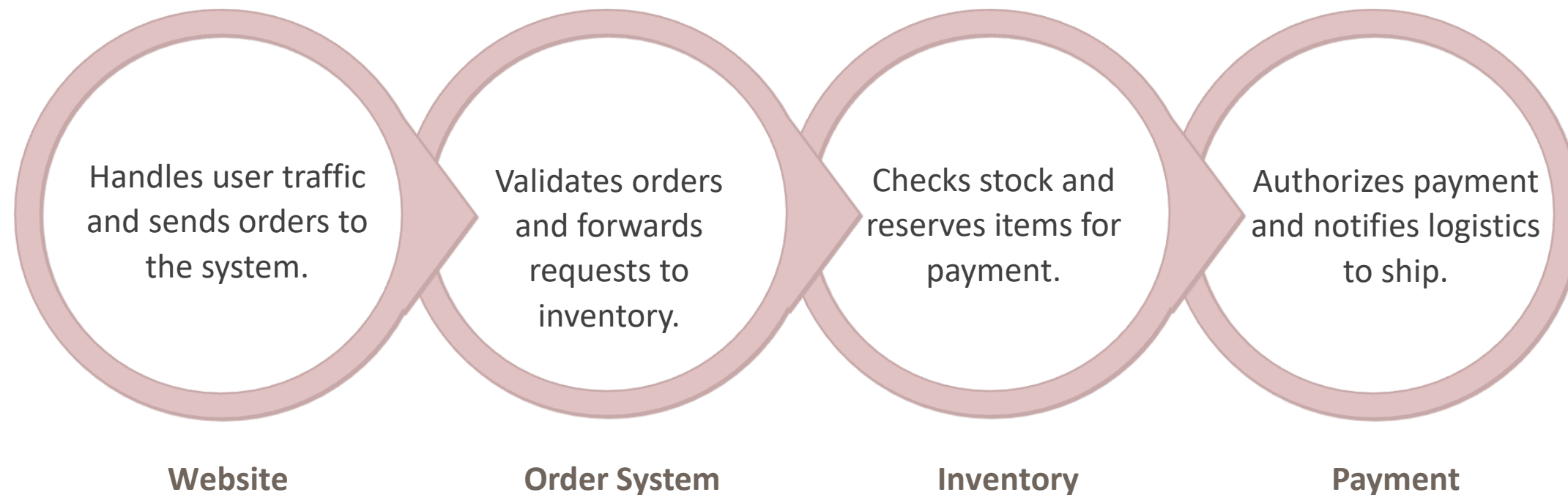


15-Minute Whiteboard Challenge

A hands-on systems thinking exercise for digital PMs: E-Commerce System Rescue Mission

- Scenario: A live e-commerce platform under extreme load. Your mission: **diagnose the crisis, redesign the architecture, and save the system — in 15 minutes.**



PHASE 1

The Bottleneck Crisis

It's **Double 11 Mega Sale** day. **10,000 orders** just flooded in — and the system is crashing.

The Point-to-Point Trap

The current system runs on a **strict linear dependency**. Each system must fully complete before the next one can begin. No **parallelism**. No flexibility. Pure bottleneck.

Two Complications Making It Worse

- **Scope Drift:** The CEO suddenly demands **5 more people** inserted into the Payment System to prevent fraud — mid-crisis.
- **WIP Explosion:** The Website Platform keeps pushing all 10,000 orders to the Order System at once, creating a **massive queue** right in front of Payment. The system cannot breathe.

⚠ The system is crashing. Orders are piling up. Every second of delay costs real money.

💬 Discussion Question 1

The project is seriously delayed. The CEO says: "Let's **hire 5 more engineers** to help the Payment System team right now!"

Will this solve the delay? Why or why not? Discuss with your team before answer.

PHASE 2

Architecture Refactoring & Agile Rescue

Forget the linear flow introduced earlier. It's time to **redesign the entire system** from scratch.

Your Challenge as Digital PM

Transform the rigid, linear model into a flexible, parallel architecture. The goal: systems **no longer wait for each other**.

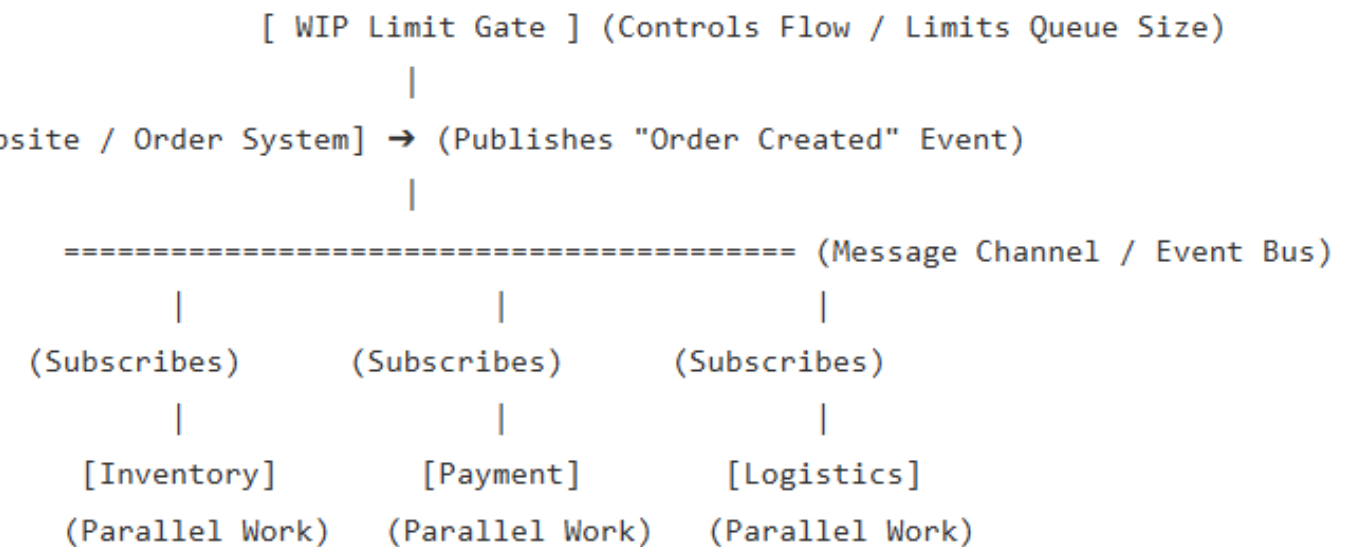
- Break the linear dependency chain
- Allow Inventory, Payment, and Logistics to work **simultaneously**
- Introduce a **WIP Limit Gate** based on Little's Law to control flow

Key Design Principles

- **Publish:** The Order System broadcasts an **"Order Created"** event to a central Message Channel
- **Subscribe:** Inventory, Payment, and Logistics each **listen** and react independently
- **WIP Gate:** Instead of pushing 10,000 orders at once, **limit how many enter the pipeline at any time**

Discussion Question 2

- How do you redraw the system dependency diagram on the whiteboard to reflect the new **architecture**?
- Where exactly should the **WIP Limit Gate** be placed, and why does it reduce Lead Time according to Little's Law?



Debriefing & Summary

Let's connect our whiteboard drawings back to the core theories. What did we actually learn?

1

The Trap of Dependencies

Hidden dependencies create invisible **waiting times**. In digital projects, **linear handoffs (dependencies)** are **the main cause of delay** — not lack of effort or talent. Every **arrow** in a linear flow is a potential bottleneck.



2

The Power of Architecture

Modular, event-driven architectures like **Publish-Subscribe** dramatically reduce coordination costs. When systems are **decoupled**, teams can process tasks in parallel — multiplying **throughput** without adding headcount.

3

The Cost of Change Curve

Fixing architecture problems **early** is cheaper. If we had waited until the final **Integration Stage** — or worse, until customer complaints triggered an External Failure Cost — causing serious budget overruns.

- ✓ The best time to fix a broken architecture is **as early as possible**. The **second best time is right now** — during the whiteboard session, not after launch.

DISCUSSIONS

Q1: Will Adding 5 More Engineers Help?

Q2: What Should the New Architecture Look Like? Please redraw the flow.